

证据驱动的科研阅读 与知识协作系统

Liteasy 问题分析、技术路线、系统设计与工程说明

让每一次理解都能回到来源，让每一次智能操作都经过受控边界，让个人阅读自然沉淀为可复用的知识。

摘要

科研阅读的困难并不只在“读懂一篇论文”，而在于把分散的文件、来源证据、解释过程、跨文献关系、个体认知和协作讨论组织成一个可持续的工作系统。Liteasy 以桌面端为主要入口，围绕“建立工作区、锁定文献、导入解析、证据检索、渐进式薄读、生成学习产物、沉淀知识与社区协作”构建闭环。系统采用 Tauri + React + TypeScript 的桌面工作台、Rust 本地能力层、Node.js 云服务、PostgreSQL、S3 API 对象存储、Keycloak 统一身份和受控模型网关，并以 IntuEcho 承担学术内容协作与文献身份权威。

技术设计的核心不是把大模型直接接入界面，而是建立三条可验证边界：其一，模型只能通过已注册的技能与动作修改系统状态；其二，论文解释必须按来源模式组织证据，并对句子、数值和图表关系逐级校验；其三，本地文件、个人云端数据、组织共享数据、平台配置与临时缓存拥有不同真源、权限和生命周期。本文给出问题分析、技术路线、系统上下文、模块与数据设计、关键流程、安全可靠性和部署与测试体系，并在附录中集中说明能力边界及后续完善项。

关键词：科研阅读；检索增强生成；证据可追溯；智能代理；生成式界面；本地优先；文献身份；多模态可视化

阅读指南

产品与业务读者

重点阅读第 1、2、3 章，理解问题、闭环和差异化价值。

研发与架构读者

重点阅读第 4 至 8 章，理解模块、契约、数据与部署边界。

评审与运维读者

重点阅读第 9 章和附录，核对质量体系、能力边界与证据依据。

目录

1. 问题分析

1.1 科研阅读的系统性矛盾

1.2 设计目标与关键约束

2. 产品闭环与设计思想

2.1 从文件到认知资产

2.2 五项架构原则

3. 技术路线

3.1 混合计算与分层技术栈

3.2 方案选择说明

4. 系统总体设计

4.1 系统上下文

4.2 容器与服务架构

4.3 桌面端模块边界

5. 智能代理与安全执行

5.1 受控动作链

5.2 生成式界面

6. 薄读、证据与多模态

6.1 渐进认知图

6.2 四档来源模式

6.3 数值与图表保真

7. 文献身份与数据设计

7.1 多层身份模型

7.2 本地与云端存储

8. 安全、部署与运行

8.1 身份、授权与隔离

8.2 预发布部署拓扑

9. 质量工程与可观测性

附录 A. 能力边界与后续完善事项

附录 B. 仓库证据与术语

参考文献

问题分析

Liteasy 面向的不是一次性的“论文问答”，而是科研用户长期处理文献、证据、理解、产物和协作关系时形成的系统性摩擦。

1.1 科研阅读的系统性矛盾



图 1 问题链与系统解法

1.2 设计目标与关键约束

目标	可观察结果	工程约束
降低进入论文的认知门槛	先给全景，再允许按兴趣逐层深入	认知图按需展开；层级是视图属性，不是永久标签
让结论可回到来源	解释句、定量主张和图表均能显示依据	来源白名单、句级覆盖、数值证明、身份确认
让 AI 能操作但不能越权	用户能用自然语言控制工作台，并看到确认与结果	模型只提计划；策略层只放行已注册动作
同时支持个人与组织	本地库不随账号变化，组织资源按角色访问	资源作用域、数据库和业务会话隔离
稳定呈现多模态产物	图、公式、图片与交互结果可显示、可降级	结构化协议、渲染器注册表、质量门与资源预算

产品闭环与设计思想

产品主线从“打开文件”升级为“建立可追溯认知资产”。每个环节都保留显式状态，使用户知道系统掌握了什么、正在做什么、结果从哪里来。

2.1 从文件到认知资产

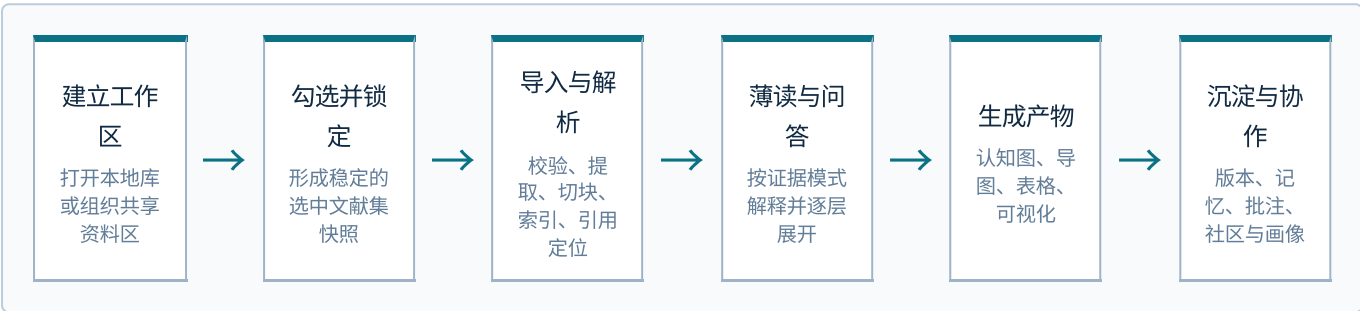


图 2 Liteasy 核心用户闭环

工作区是“当前可见的目录视图”，选中文献集是“用户勾选并锁定、准备交给智能能力处理的文件快照”。二者刻意分离：目录可以继续变化，而一次分析必须基于稳定输入。导入不是简单上传，而是对文献执行安全校验、文本与结构提取、切块、索引和定位准备。分析结果以产物及版本保存，能够被再次打开、继续下钻或与社区内容关联。

2.2 五项架构原则

证据优先 论文事实、外部知识、社区表达、用户笔记与 AI 独立理解分别标注，不能用流畅措辞掩盖来源差异。	动作受控 按钮、自然语言和快捷入口复用同一动作注册表；权限、风险和确认在执行前统一处理。	本地优先 本地文件与私有阅读细节以设备为权威；云端承担协作、同步、推荐、任务与治理。
协议驱动 模型输出进入类型化协议和校验管线，界面不执行任意模型代码，数据也不依赖自由文本猜测。	失败可恢复 长任务有状态，写入具备幂等键和版本检查，文件操作使用原子提交或可恢复事务。	渐进展开 先呈现少量核心节点，再随用户兴趣加载细节，以认知负担和证据密度共同决定深度。

一句话架构思想

大模型负责提出解释和计划，确定性系统负责决定“能不能做、依据是否成立、结果能否保存”。

Liteasy 采用“桌面重交互 + 本地文件能力 + 云端业务与任务 + 外部模型能力”的混合路线，在隐私、体验、可扩展性和模型迭代速度之间取得工程平衡。

3.1 分层技术栈



图 3 技术路线分层图

3.2 关键选型说明

领域	主要技术	选择依据	设计收益
桌面宿主	Tauri 2 + Rust	需要安全访问文件系统、系统密钥环、OAuth 回调和原生窗口	安装体积与运行开销较低；文件能力与 Web 界面解耦
工作台界面	React + TypeScript + Fluent 2	复杂三栏布局、标签页、可访问控件与高频状态反馈	组件生态成熟，类型契约可贯穿界面与运行时
文献解析	PDF.js、GROBID、MinerU、Tesseract	文本层、学术结构、图像与 OCR 需要多路径互补	按文献质量选择路径，并保留降级方案

领域	主要技术	选择依据	设计收益
可视化	Mermaid、KaTeX、XYFlow、D3、Three.js	流程、公式、认知图与三维对象具有不同渲染语义	统一产物协议下按类型选择专用渲染器
正式数据	PostgreSQL 15+ + S3 API	需要事务化逻辑树、权限、版本、对象去重和备份	元数据与大对象分离；逻辑归属与物理内容解耦
身份与服务	Keycloak、OIDC/JWKS、Node.js 20+	桌面、论坛、管理端和服务间调用具有不同受众	共享身份源而不共享业务会话、数据库或凭据

3.3 架构报告方法

本文借鉴 C4 Model 的分层视图方法，以“系统上下文、容器、组件、动态流程、部署”逐步展开架构，使业务读者和研发人员共享同一套图形语义 [1]。质量属性参考 Google Cloud、AWS 与 Microsoft 的良好架构框架，将安全、可靠性、性能效率、成本与运维卓越作为横切关注点 [2-4]；运行质量参考 Google SRE 对服务目标、监控、数据完整性、发布工程和简洁性的组织方式 [5]。这些资料用于报告结构与评审维度，不替代项目自身的代码和设计事实。

系统总体设计

总体设计以资源所有权和执行权限为边界：桌面端拥有本地交互与文件入口，Liteasy 云服务拥有个人与组织业务数据，IntuEcho 拥有文献身份和社区内容，身份平台提供统一主体。

4.1 系统上下文



图 4 系统上下文图

4.2 容器与服务架构



图 5 容器与服务边界

两个业务产品共享身份主体，但不共享数据库、连接池、凭据或用户会话。Liteasy 需要文献身份时，以固定服务身份调用 IntuEcho 的内部接口，并保存已核验的只读投影。对象存储中的 PDF 字节可以按内容哈希去重，但任何下载都先验证调用者对逻辑文献条目的权限，不能凭哈希直接获取对象。

4.3 桌面端模块边界

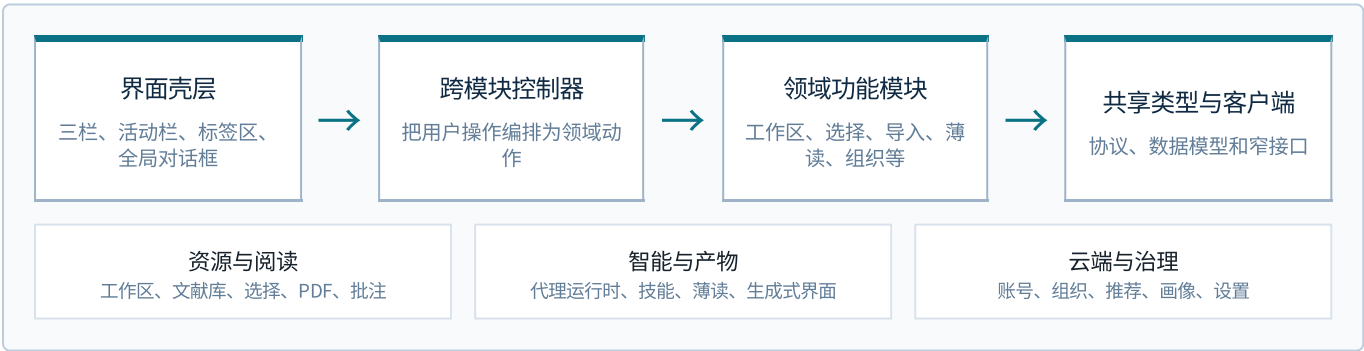


图 6 桌面端依赖方向与功能分组

依赖方向固定为“界面壳层 → 控制器 → 功能模块 → 共享类型/客户端”。界面壳层只组合区域，不承担跨区域复制、权限判定或文件事务；功能模块也不得反向依赖壳层。标签页由 Dock 子系统管理展示位置和布局树，关闭或移动标签只改变视图，不删除领域数据。这个边界减少了大型工作台常见的全局组件膨胀和状态耦合。

仓库规模统计与可复现口径见附录 B；相关数字仅用于说明工程规模，不代表覆盖率、性能或生产成熟度。

智能代理与安全执行

Liteasy 的智能代理是工作台的统一交互入口，但不是绕过应用规则的超级权限。它把自然语言目标转换为可校验计划，再通过受控动作执行。

5.1 受控动作链

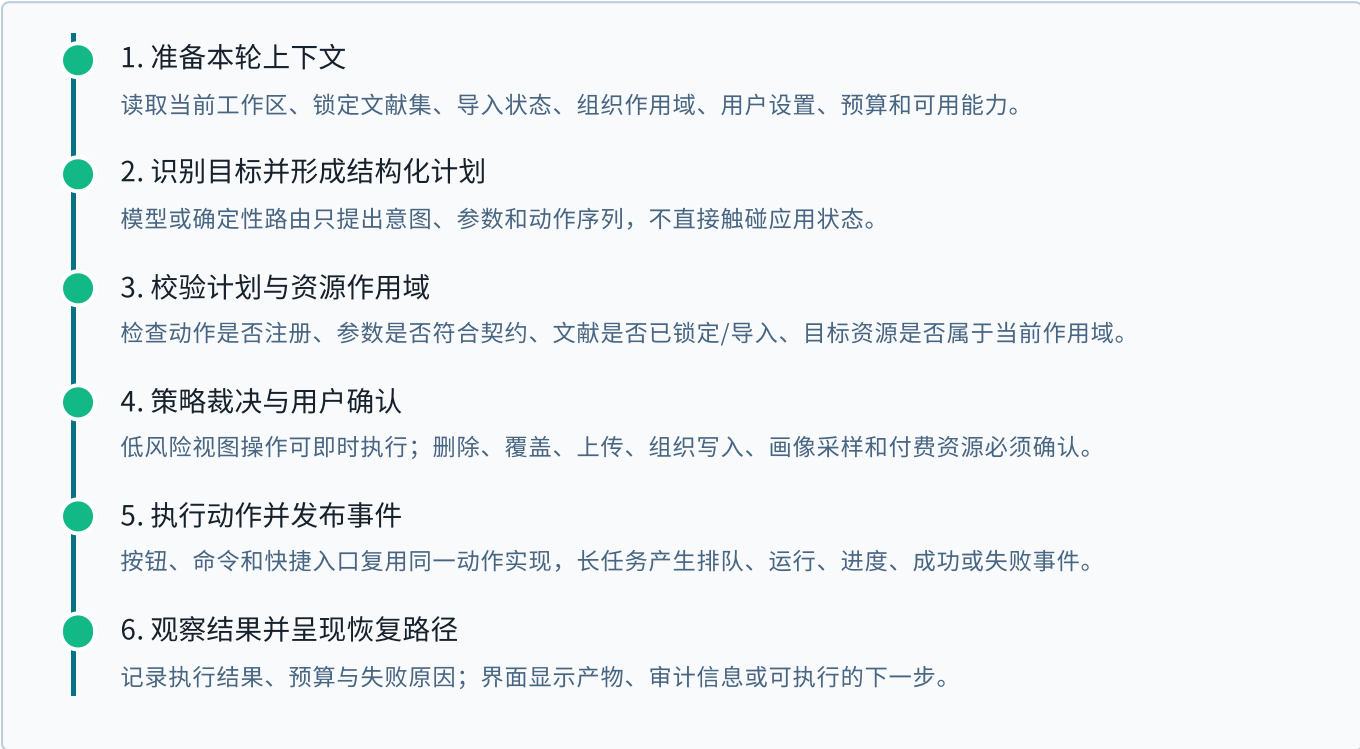


图 7 一次智能操作的动态流程

风险等级	典型动作	执行方式	必须记录
低	切换标签、调整面板、打开设置	即时或乐观执行	动作名、参数、结果
中	生成产物、同步元数据、启动外部检索	显示任务状态，可取消或重试	任务、来源、成本、产物版本
高	删除、覆盖、上传、组织写入、画像采样	执行前明确确认，服务端再次授权	主体、作用域、确认、幂等键、审计事件

5.2 技能、动作与插件边界

技能描述“如何完成一类任务”，可以组合检索、生成和动作；动作描述“系统允许改变的一个确定状态”，具有类型化参数、风险、作用域和执行器。插件只能通过公开能力注册点扩展技能、界面项或受控动作，不能获

得任意 DOM、文件系统或服务凭据访问。由此形成一条清晰的权限路径：

用户目标 → 技能选择 → 动作计划 → 契约校验 → 策略与确认 → 确定性执行器

任何模型输出在进入执行器前都不是事实，也不是授权。

5.3 生成式界面

生成式界面不让模型返回 React、CSS 或脚本。模型或规则只能生成受约束的界面描述文档，指定已注册组件、已注册数据源、允许的动作和设计令牌。动态画布负责解释该文档；结构校验器保证协议正确，体验校验器检查布局、可访问性与操作风险，执行日志记录用户真正触发的动作。

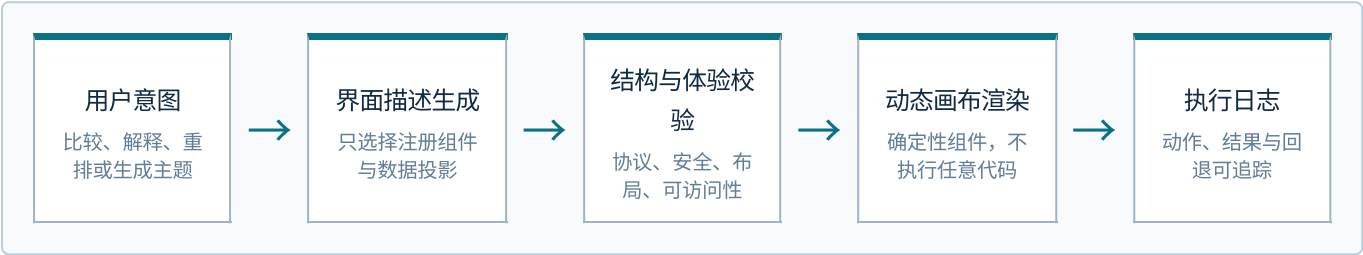


图 8 受约束生成式界面链路

薄读、证据与多模态

“薄读”不是把论文压缩成更短的摘要，而是先构建低认知成本的全景，再允许用户沿概念、机制、实验或疑问逐层加深，同时持续维护来源与证明关系。

6.1 渐进认知图

认知结构采用有向多重图。只有“从略到详”的展开关系要求无环，解释、支持、反驳、先修、比较、来源和直觉关系可以交叉连接。领域事实保存在规范化认知图中，界面描述只是对图的投影；用户每次展开只提交带基础版本号的局部补丁，避免整图覆盖和并发丢失。

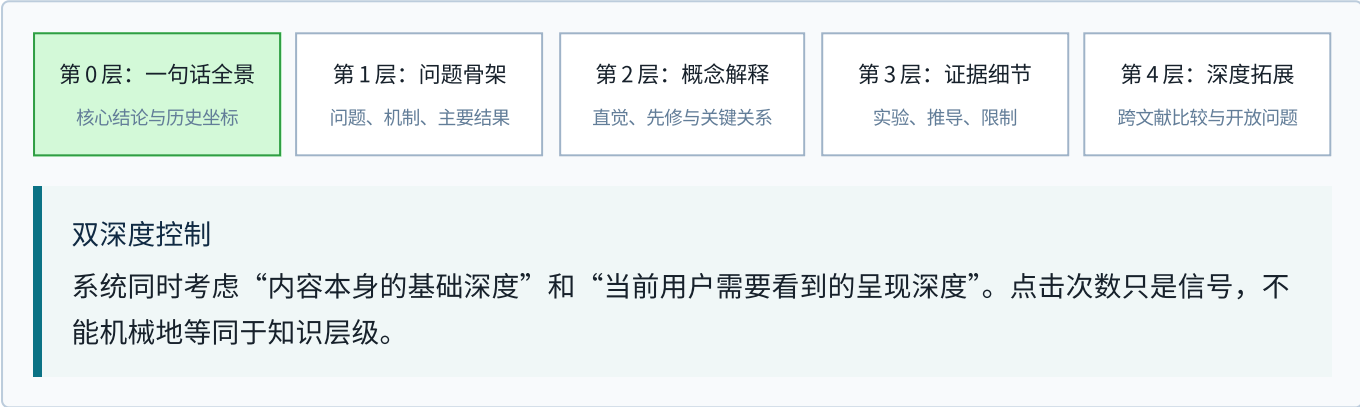


图 9 渐进式认知投影

6.2 四档来源模式

模式	正文可以使用的依据	适用问题	界面表达
论文内证据 paper	目标论文中可定位的文本、表格、图和结构	“作者做了什么”“实验结果如何”	显示论文来源和锚点
论文 + 外部证据 paper_and_external	论文内证据与经验证的外部学术来源	背景补充、支持/挑战、跨文献比较	分别标注论文内外来源
仅外部证据 external_only	经验证的论文外来源；本地索引只恢复任务范围	论文未覆盖但可从外部文献回答的问题	明确“不依据目标论文正文”
AI 独立理解 ai_interpretation	不声称具有文献依据的模型理解	可信外部来源耗尽后的通用解释	显著标注无文献依据

6.3 统一生成与质量门

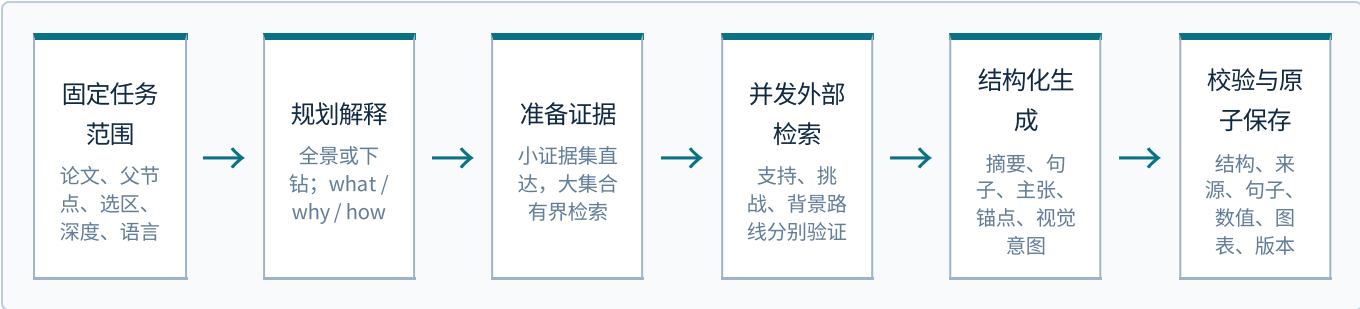


图 10 薄读统一生成流水线

生成前固定目标论文和上下文；证据准备阶段对大集合采用“轻量目录规划 → 白名单工具读取 → 最多一轮补充”的有界循环；外部来源按支持、挑战和背景路线并发检索，单路失败不拖垮其他可信来源。正文以严格结构化对象返回，流式内容只作为草稿展示。只有结构、来源模式、句级覆盖、定量命题和语义复核全部通过后，节点与产物版本才会原子保存。

6.4 数值与图表保真

数值保真门 每个正文定量主张都要有合法证明路径，逐项匹配主体、指标、关系、值、单位、比较对象、限定词、不确定性与实验条件。	数值完整性门 仅在用户要求“列全结果”或任务定义必需事实集合时启用，避免把合理省略误判为错误。	图表结构门 仅在存在结构化图表证据时验证行列、图例、坐标和区域关系；数字相邻不能被推断为表格关系。
---	---	---

数值证明坚持原子性：默认由同一来源、同一实验范围内的事实支持一个经验主张，禁止跨论文、跨硬件、跨数据集或跨统计口径拼接新比较。失败修复遵循“确定性规范化 → 删除非正文可选字段 → 只改失败句 → 隔离不合格句 → 定向换源 → 合法切换来源模式”的顺序，已经通过的句子和来源映射保持冻结。

6.5 多模态产物

系统把多模态生成拆为“意图、证据、产物协议、编译器、渲染器、校验器”六个角色。语义图、电路与物理图、生物结构、函数与几何、过程动画、栅格插图分别使用专用编译器；界面根据设备能力和产物状态选择 SVG、Canvas、WebGL、图片或无障碍文本降级。自动视觉增强与正文拥有独立预算，因此附属图像失败不会破坏已经合格的正文。

文献身份与数据设计

科研系统必须同时回答“这是哪个文件副本”“字节是否相同”“它对应哪个已确认学术版本”“版本之间是什么关系”。Liteasy 为这些问题分配不同标识和权威服务。

7.1 多层文献身份



图 11 文献身份分层

IntuEcho 的独立文献模块是来源解析、分级裁决和确认记录的权威。原始注册来源（如 Crossref DOI、arXiv）可按其稳定标识精确回查；聚合来源（如 OpenAlex、Semantic Scholar）需要两个独立来源一致，或由用户选择后服务端重新抓取验证。状态从“未解析 → 候选 → 已确认”演进，歧义和冲突进入独立分支。只有已确认记录才能作为公开批注、社区同步和跨用户精确推荐的文献范围。

7.2 本地与云端存储边界



图 12 本地与云端数据边界

本地库属于当前操作系统用户，不随 Liteasy 登录账号切换；同一时刻启用一个库根目录，文件系统是存在性的真源。云端使用 `scope_type + scope_id + folder_id` 表达个人或组织归属和目录层级。PDF 写入临时对象

时由服务端计算 SHA-256、执行安全扫描，再在数据库事务中创建逻辑文献条目和对象引用；相同字节可以复用对象，但配额仍按逻辑文献正文计算。

7.3 资源操作语义

资源区	真源与生命周期	跨区域拖拽语义	权限要点
本地文献库	设备真实目录；用户负责备份	向云端上传副本，来源不删除	路径必须限制在库根目录内；默认不跟随符号链接
个人收藏	云端长期数据；进入配额、备份与账号生命周期	下载到本地或复制到有权限的组织	仅用户本人或明确授权流程可修改
关联推荐	带期限的虚拟结果；不是永久目录	保存 PDF 或仅元数据条目	个性化关闭后仍可按显式文献/检索词推荐
组织文献库	组织作用域长期数据	导入与导出均受组织策略控制	拖拽开始和提交时都重新校验角色与策略

安全、部署与运行

安全设计贯穿身份、文件、模型、对象和界面五个面向。预发布环境采用单一 HTTPS 入口与内部网络隔离，并把数据库、对象存储和扫描服务作为就绪条件。

8.1 身份、授权与隔离

统一身份，不共享业务会话 桌面、社区和管理端使用不同 OIDC 客户端与令牌受众。管理登录具有独立认证流和多因素要求。	服务端是最终授权点 前端禁用控件只改善体验；用户作用域来自签名令牌，组织成员与存储策略由数据库重新判断。
上传链路故障关闭 PDF 限制大小和类型，经私有 HTTPS 扫描并绑定内容哈希；扫描不可用时拒绝上传。	模型密钥与路由固定 上游模型、密钥和能力由部署配置决定，桌面客户端不能覆盖；输出仍要经过协议与质量门。
路径与对象防越界 本地命令拒绝路径穿越和危险重解析点；云端下载先验证逻辑条目权限，再解析对象引用。	最小化个性化数据 允许上传必要题录和显式行为，不上传绝对路径、用户名、无关目录、私密批注或任意文件内容。

上述控制与 NIST AI 风险管理框架的“治理、映射、测量、管理”循环一致 [6]，并针对 OWASP 大模型应用风险中的提示注入、不安全输出处理、敏感信息泄露、过度代理权限和无界消耗设置确定性边界 [7]。

8.2 预发布部署拓扑

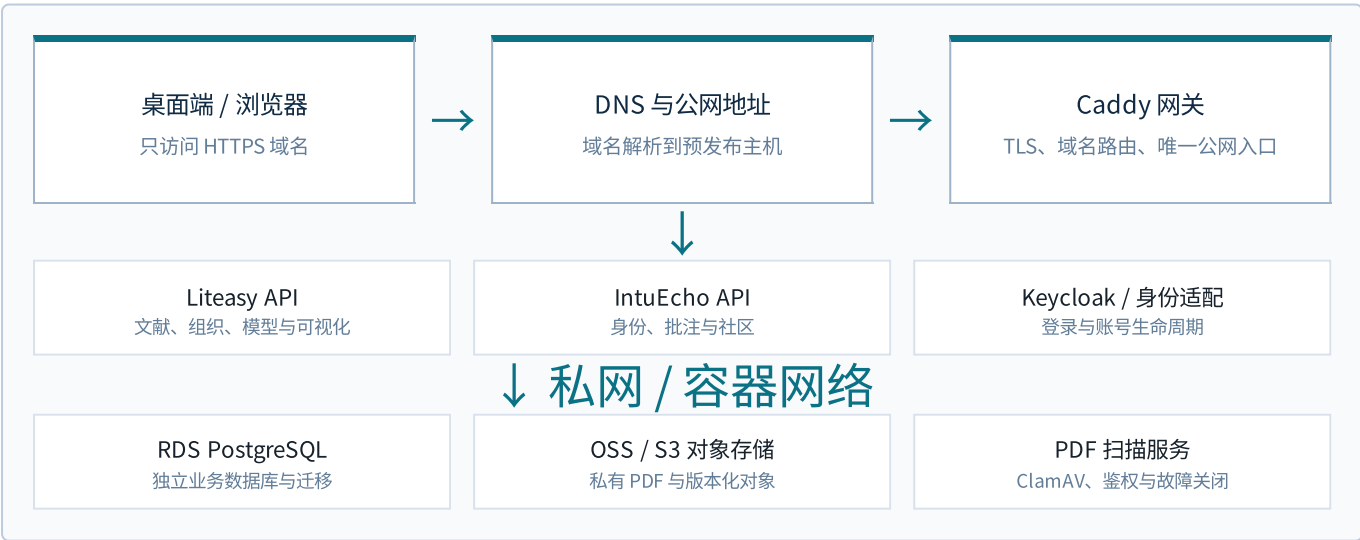


图 13 预发布部署视图

公网只开放 HTTP/HTTPS，应用内部端口不映射到互联网。服务在监听端口前检查数据库迁移、对象存储公共访问阻断、加密和版本化等条件；迁移由独立账号执行并使用 advisory lock，已执行迁移的内容哈希变化会阻止启动。存活检查回答“进程是否运行”，就绪检查回答“数据库、身份和存储是否满足接流量条件”，两者分开设计。

8.3 事务、幂等与恢复

场景	一致性机制	恢复策略
本地文件导入/移动/删除	分块写入、原子发布、操作标记、索引修订	重启时识别已准备/已提交事务并回滚或完成
云端 PDF 上传	临时对象 + 服务端哈希 + 扫描 + 数据库事务	失败清理临时对象；周期回收无引用对象
认知图局部扩展	基础版本号、幂等键、事务化产物版本	版本冲突时重新装载和变基，不使用最后写入覆盖
账号删除与画像清除	幂等工作流、作用域删除与版本失效	记录阶段状态，重复调用继续未完成步骤
长时生成任务	任务状态机、预算、超时、断路器	保留已通过正文；附属增强独立重试或降级

质量工程与可观测性

系统质量不是单次模型得分，而是从确定性契约、领域校验、模型评测、集成验证到发布门禁构成的证据链。

9.1 分层质量体系



图 14 质量工程金字塔

9.2 关键观测对象

观测域	关键事件/指标	回答的问题
交互与运行时	路由耗时、上下文装配、确认率、动作成功率、恢复率	用户目标是否被快速、正确地转为受控操作？
检索与证据	首结果时间、来源覆盖、验证失败、换源次数、句级支持率	回答是否建立在足够且合法的来源上？
生成与产物	首状态、首屏、完成耗时、修复次数、降级类型、版本冲突	长任务是否及时反馈并稳定交付？
数据与存储	事务失败、孤立对象、垃圾回收、备份与恢复演练	读取的数据是否等于已确认写入的数据？
安全与身份	受众拒绝、授权拒绝、扫描失败、管理员 MFA、支持访问审计	主体是否只在被授权的作用域内行动？
成本与容量	模型调用、证据数量、视觉计算单元、队列、内存与并发	体验、质量和成本是否处在可解释预算内？

9.3 服务目标方法

在冻结端到端毫秒级承诺前，系统先建立可重复基线：将本地命令、上下文装配、检索首结果、任务首状态、产物首屏和模型完成分别测量；把模型质量与基础设施可用性分开；以真实用户路径和固定评测集观察变化。该方法符合 SRE 对服务级目标、监控、测试可靠性、发布工程和数据完整性的强调 [5]。任何性能数字都应同时给出数据集、硬件、网络、并发和版本条件。

工程完成定义

代码存在不是完成；必须同时具备契约、测试、失败恢复、可观测事件、部署配置和可复现验收证据。

能力边界与后续完善事项

本附录集中说明仓库快照中的能力状态与后续工程事项，避免在正文架构叙事中混淆“设计成立”和“目标环境已经验收”。

A.1 能力成熟度说明

领域	仓库快照中的基础	后续完善重点
桌面工作台	已形成可运行骨架：三栏、Dock、工作区、锁定选择、PDF、助手、产物、组织、设置	持续收敛跨模块控制器，扩大真实设备端到端验证与可访问性回归
本地文献库	真实文件链已实现：目录扫描、监听、分块导入、原子写、回收站、恢复与备份	面向更多 Windows 文件系统差异、超大库容量和长期迁移开展压力与恢复演练
智能代理	运行时与安全边界已建立：上下文、计划校验、策略、确认、技能、动作、任务事件	扩充跨技能评测、持久记忆治理、正式插件权限与 MCP 注册生命周期
薄读与多模态	协议和多类渲染链存在：四档来源、数值保真、静态科学、数学、过程栅格与发布门	继续扩大真实论文金集、跨学科图表语义评测、提供方容量和成本基线
正式云服务	生产边界代码已建立：PostgreSQL/S3、OIDC、扫描、组织、推荐、模型代理、账号生命周期	补齐目标环境的托管密钥、备份恢复、容量、监控告警和完整上线证据
身份与社区	独立服务边界已建立：来源确认文献身份、社区内容、批注、关系、服务间授权	持续验证外部来源波动、审核申诉、删除传播与社区规模下的排序质量
发布与运维	受控预发布链可用：Caddy、Compose、Keycloak、扫描器、Windows 安装包交接	生产签名、公开发布、扩容、RDS 恢复、真实告警和无人值守故障演练

A.2 当前预发布边界

- 当前部署资料定义的是最多 10 个具名受邀账号、低并发的受控预发布，不等同于公开生产服务。
- 现有 Windows 安装包在负责人批准下用于受控预发布，Authenticode 未签名；生产和不受控公开分发需要完成代码签名。
- 预发布主机基线为 2 vCPU / 4 GiB；扩大账号、PDF 上限、模型并发或公开测试前需要扩容并重新验收。
- development/dev-cloud 是本地真实联调服务，使用 SQLite 与本地对象目录，并在 staging/production 环境拒绝运行；它不应被描述为正式云平台。
- 完整生产能力的声明必须以目标环境中的数据库、对象存储、密钥、扫描、备份、恢复、监控、告警和端到端证据为准。

A.3 建议优先级

- P0：发布可信链
完成安装包签名、目标环境托管密钥、备份恢复与监控告警，以证据关闭生产门禁。
- P0：评测可信链
固定跨学科论文金集、证据/数值/图表指标和回归阈值，形成模型与渲染器版本对比。
- P1：长期记忆治理
本地持久化、命名空间隔离、用户预览/删除/导出、写入过滤、命中解释与污染评测。
- P1：规模与成本基线
在真实并发、超大文献库和多模态任务下测量延迟、队列、内存、存储与模型计算单元。
- P2：生态扩展
在动作、权限、审计和生命周期协议稳定后开放插件与 MCP，避免扩展先于治理。

仓库证据、术语与编制说明

B.1 主要内部事实源

主题	主要文档或代码位置
项目总览与实现边界	docs/□□□□□□□□□□.md； docs/qa/2026-08-08-repository-structure-and-capability-audit.md
产品蓝图与 SaaS 定位	docs/superpowers/specs/2026-05-10-liteasyclaw-product-blueprint-design.md； docs/saas/LiteasyClaw_SaaS□□□□□□□□□□.md
桌面模块与代理	docs/engineering/module-boundaries.md； docs/agent-dev/liteasy-agent-core-design.md； products/liteasy/apps/desktop/src/app/features/
薄读与多模式	docs/design/□□-□□□□□□□□□□□□□□.md； docs/design/□□-□□□□□□□□□□□□□□.md； docs/superpowers/specs/2026-08-08-thin-reading-multimodal-visualization-design.md
文献身份与存储	docs/design/□□□□□□□□□□□□□□.md； docs/design/Liteasy-□□□□□□□□□□□□□□.md
正式服务与部署	products/liteasy/services/api/README.md； deployment/staging/README.md； docs/operations/staging-handoff-2026-08-16.md

B.2 仓库快照统计口径

统计基于 Git 提交 54cc18a。桌面功能文件为 products/liteasy/apps/desktop/src/app/features 下全部普通文件；桌面测试文件为 src/tests 下匹配 *.test.* 的文件；Tauri 命令为 Rust 源码中 #[tauri::command] 标记数量；源码行数统计桌面 TypeScript/TSX、Rust 宿主、Liteasy API、IntuEcho API 和身份服务中的 TypeScript/TSX/Rust/MJS 文件。统计未排除空行或注释，因此仅表示可复现的仓库规模快照。

B.3 术语表

术语	本文含义
工作区	当前可见的本地或组织文献目录视图；切换工作区不是合并资源。
选中文献集	用户在当前工作区勾选并锁定、准备交给智能能力处理的文件快照。
导入	对选中文献执行校验、解析、切块、索引和引用定位准备。
产物	具有类型、来源、版本和生命周期的分析结果，如认知图、导图、表格或可视化。
技能	完成一类用户任务的受控流程，可以组合检索、生成和动作。

术语	本文含义
动作	一个可以确定性执行的状态变更，带类型化参数、资源作用域、风险与权限。
证据锚点	连接解释与来源的定位信息，可包含页码、文本范围、结构区、坐标或内容哈希。
薄读	先建立低认知成本全景，再沿用户关注点逐层展开的证据约束阅读方式。

B.4 编制与学术规范

本报告以项目仓库的设计文档、实现代码、测试结构、部署手册和交接证据为内部事实源；以公开的官方架构框架、安全规范和学术基础文献为外部参考。正文没有把规划、代码存在或确定性规则测试等同于生产环境验收或真实模型质量；相关边界集中列于附录 A。图表均为基于仓库事实重新绘制的中文解释图，不复制外部厂商图形资产。

参考文献

1. Brown, S. *The C4 model for visualising software architecture* [EB/OL]. <https://c4model.com/> (访问日期: 2026-08-17).
2. Google Cloud. *Google Cloud Well-Architected Framework* [EB/OL]. <https://cloud.google.com/architecture/framework> (访问日期: 2026-08-17).
3. Amazon Web Services. *AWS Well-Architected Framework* [EB/OL]. <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html> (访问日期: 2026-08-17).
4. Microsoft. *Azure Well-Architected Framework* [EB/OL]. <https://learn.microsoft.com/azure/well-architected/> (访问日期: 2026-08-17).
5. Beyer, B., Jones, C., Petoff, J., Murphy, N. R., eds. *Site Reliability Engineering: How Google Runs Production Systems* [M/OL]. O'Reilly Media, 2016. <https://sre.google/sre-book/table-of-contents/> (访问日期: 2026-08-17).
6. National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)* [R]. NIST AI 100-1, 2023. doi:10.6028/NIST.AI.100-1.
7. OWASP Foundation. *OWASP Top 10 for Large Language Model Applications* [EB/OL]. <https://owasp.org/www-project-top-10-for-large-language-model-applications/> (访问日期: 2026-08-17).
8. Lewis, P., Perez, E., Piktus, A., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [C]//*Advances in Neural Information Processing Systems*. 2020, 33: 9459-9474.
9. Crossref. *Relationships: Markup guides for metadata segments* [EB/OL]. <https://www.crossref.org/documentation/schema-library/markup-guide-metadata-segments/relationships/> (访问日期: 2026-08-17).
10. W3C. *PROV-O: The PROV Ontology* [EB/OL]. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-o/> (访问日期: 2026-08-17).
11. Fielding, R. T. *Architectural Styles and the Design of Network-based Software Architectures* [D]. University of California, Irvine, 2000.
12. ISO/IEC/IEEE. *ISO/IEC/IEEE 42010:2022 Software, systems and enterprise — Architecture description* [S]. Geneva: ISO, 2022.